

Connected Vehicle Intelligence System

Secure Communication Architecture for Cloud-Centric Automotive AI

[Your Name(s)]

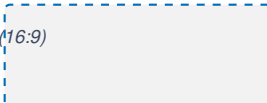
Academic Year 2025–26

Department of [Your Department]

SRM Institute of Science and Technology

Computer Communication & Network Security (CCNS)

Hero Image (16:9)



Primary Objective & Core Thesis

Core Thesis

AI should live **centrally** (cloud / data-centre), not embedded per-vehicle.

The vehicle is a **thin, securely-networked client**.

Primary Objective

- ▶ Demonstrate the **secure communication layer** that makes this architecture possible.
- ▶ Map every feature to **CCNS Units 1–5** (graded surface).
- ▶ AI is the *application riding on top* — not the point of the project.

*Concept Diagram:
Heavy Edge vs
Thin Client + Cloud AI*

Problem Statement & Scope

Problem Statement

- ▶ Per-vehicle edge AI is **expensive & hard to update**.
- ▶ Centralising AI exposes traffic to:
 - Packet loss & high latency
 - Payload tampering (MITM)
 - Unauthorised device access

Project Scope

In Scope:

Simulated ESP32, HTTP+MQTT, HMAC/AES-GCM, chaos middleware, Next.js UI.

Out of Scope:

*Threat Model Diagram:
Vehicle Network path
with MITM / Replay /
Tamper threat icons*

System Abstract & Architecture

Four-tier architecture: ESP32 edge node → Secure comms → FastAPI + Ollama AI → Unified frontend

*Architecture Diagram (four tiers, left-to-right):
[ESP32] -> [Secure Comms HTTP/MQTT] -> [FastAPI + Ollama AI] -> [Next.js /driver /noc /admin]
Labeled arrows: Signed JSON, HMAC Verified, WebSocket Events*

Tech Stack:

ESP32 (Arduino) • Python FastAPI • SQLite • Ollama llama3.2:3b • Next.js + Tailwind

Methodology: System Design & Interfaces

Protocol Adapters (Runtime-switchable)

- ▶ `HttpAdapter` — POST to `/api/v1/telemetry`
- ▶ `MqttAdapter` — `paho-mqtt` ON `cvis/telemetry/+`
- ▶ Switch via POST `/api/v1/config/protocol`

Ingest Pipeline (Executed for every packet)

1. ChaosMiddleware (drop / delay / tamper)
2. Auth layer (API-key validation)
3. HMAC-SHA256 verify (constant-time)
4. AES-256-GCM decrypt (if encrypted)
5. Pydantic validation
6. SQLite persist
7. WebSocket broadcast
8. Ollama AI task (fire-and-forget)

Ingest Sequence Diagram:
ESP32 -> Chaos -> Auth -> Service
-> SQLite / WebSocket / Ollama

Methodology: ESP32 Firmware (Edge Node)

ESP32 Board Photo

Edge Security (mbedTLS)

- ▶ HMAC-SHA256 signing
- ▶ AES-256-GCM encryption
- ▶ Hardware RNG IV
- ▶ Ack + retry backoff

8 Vehicle Modes (Button-cycled, GPIO ISR)

Mode	Telemetry Signature
Healthy	Normal across all fields
Eco	Low speed, slow drain
Sport	High temp + speed, fast drain
Heavy Traffic	Low speed, high battery use
Low Battery	Range drops, fault code set
Overheating	Motor temp critical, range drops
Charging	Speed=0, battery rising
Motor Fault	Fault code, degraded output

Security Mechanisms

- ▶ **Auth:** API key per device (SHA-256 hashed).
- ▶ **Integrity:** HMAC-SHA256, constant-time verify.
- ▶ **Encryption:** AES-256-GCM, fresh random IV.
- ▶ **Tamper:** Chaos mutates body → HMAC fail → 401.
- ▶ **Replay:** (device_id, timestamp) dedup.
- ▶ **Revocation:** Disconnect sets `active=0`.
- ▶ **Audit:** All events logged in `auth_logs` table.

Crypto Flow Diagram:
Payload -> HMAC-SHA256
-> AES-256-GCM
-> Verify on server

Methodology: Local AI Integration

Ollama AI Layer

Model: llama3.2:3b or phi3-mini

Reason: Low latency for live demo

Multi-Factor Reasoning

- ▶ Evaluates **all fields simultaneously**: battery, temp, mode, speed, range, faults.
- ▶ No `if/else` thresholds — LLM reasoning.
- ▶ **Auto-recommendation** per telemetry update.
- ▶ **Driver chat** grounded in current telemetry.
- ▶ AI kill-switch — secure comms stays independent.

*AI Comparison Card:
Bad: "if temp > 90: alert()"
vs CVIS: Multi-factor natural language*

AI Chat UI Mockup (screenshot)

Methodology: Unified Frontend Architecture

Next.js + Tailwind CSS • One app • One WebSocket • Three role-gated views

/driver

*/driver Screenshot:
Battery, Health,
AI Chat, Recharts*

Vehicle health

AI chat • Graphs

/noc

*/noc Screenshot:
Live packet flow,
Control panel*

Packet flow

Live controls

/admin

*/admin Screenshot:
Fleet stats, Auth logs,
CPU/memory*

Logs

System stats

Fleet •

Testing & Execution Plan

8-Phase Gated Build Plan

Ph.	Focus	Status
1	HTTP pipeline + DB	Done
2	Auth + HMAC + MQTT	Done
3	Ollama AI	Done
4	NOC frontend	WIP
5	Admin console	WIP
6	Hardening + e2e matrix	Pending
7	Documentation pack	Done
8	Demo rehearsal	Pending

Chaos Middleware Testing

- ▶ Packet loss: 0 / 5 / 10 / 25%
- ▶ Artificial latency: 0 / 100 / 300 / 1000 ms
- ▶ Tamper injection: mutate byte → HMAC fail

*Gantt Chart: 8-Phase Timeline
(8-week horizontal Gantt)
Blue=done, Orange=WIP, Grey=pending*

Expected Outcomes: Functional Features

- ▶ **Live packet flow** with protocol-colored animations.
- ▶ **Real-time tamper detection & rejection** (HMAC-SHA256).
- ▶ **Runtime protocol switch** HTTP ↔ MQTT, no restart.
- ▶ **Coherent multi-factor AI** across all 8 vehicle modes.
- ▶ **Resilience demo:** MQTT reconnect, ESP32 WiFi drop, AI kill-switch.
- ▶ **Full audit trail:** Auth failures, tamper events, revocations in `/admin`.

*Tamper Detection Flow:
Packet -> Byte mutated
-> HMAC fail -> 401 Rejected*

*NOC Controls Screenshot:
Chaos sliders + Protocol toggles*

Expected Outcomes: Assessment & Deliverables

CCNS Alignment

Unit	Features
U1 Data Comms	5
U2 Network Models	6
U3 Transport Layer	7
U4 Network Security	10
U5 Advanced/Cloud	6
Total	34

Deliverables

- ▶ Firmware, Backend, AI, UI
- ▶ SQLite schema + logs
- ▶ Architecture + Packet diagrams
- ▶ API docs, Manuals
- ▶ Testing & Security analysis
- ▶ IEEE-style report + PPT
- ▶ Demo script + Viva Q&A
- ▶ CCNS mapping table

Thank You.

Questions?